

SQL For SDET

Part 2: Intermediate & Advanced

★ Topics 19 to 37 — Joins, Advanced Queries & More ★



Kalpesh Jain

SDET @ Algebrik AI | Mentor | 42K+ Community ■

Bengaluru, Karnataka, India

[linkedin.com/in/kalpeshnjain09](https://www.linkedin.com/in/kalpeshnjain09)

JSPM University Pune

SQL Joins	INNER JOIN	LEFT JOIN	RIGHT JOIN
FULL OUTER JOIN	SELF JOIN	CROSS JOIN	GROUP BY
HAVING Clause	Aggregate Functions	UNION vs UNION ALL	Subqueries
EXISTS vs IN	CASE Statement	Views in SQL	Indexes in SQL
Stored Procedures	Transactions	COMMIT / ROLLBACK	

Introduction

This is **Part 2** of the SQL For SDET Complete Guide covering **Topics 19–37**. These are the intermediate and advanced SQL topics asked in product company interviews, senior SDET rounds, and backend testing discussions.

Complete Part 1 (Topics 1–18) before studying these topics.

Topic Group	Coverage
SQL Joins (6 types)	INNER, LEFT, RIGHT, FULL, SELF, CROSS
Grouping & Aggregation	GROUP BY, HAVING, Aggregate Functions
Advanced Queries	UNION, Subqueries, EXISTS vs IN, CASE, Views
Performance & Reusability	Indexes, Stored Procedures
Transactions	Transactions, COMMIT, ROLLBACK, SAVEPOINT

Table of Contents

PART 2 — INTERMEDIATE & ADVANCED (Topics 19–37)

- 19. SQL Joins ★
- 20. INNER JOIN ★
- 21. LEFT JOIN ★
- 22. RIGHT JOIN ★
- 23. FULL OUTER JOIN ★
- 24. SELF JOIN ★
- 25. CROSS JOIN ★
- 26. GROUP BY Clause ★
- 27. HAVING Clause ★
- 28. Aggregate Functions ★
- 29. UNION vs UNION ALL ★
- 30. Subqueries ★
- 31. EXISTS vs IN ★
- 32. CASE Statement ★
- 33. Views in SQL ★
- 34. Indexes in SQL ★
- 35. Stored Procedures ★
- 36. Transactions in SQL ★
- 37. COMMIT / ROLLBACK ★

SQL JOINS

19. SQL Joins ★

Overview

SQL Joins are one of the MOST IMPORTANT SQL concepts for combining data from multiple tables, backend validation, API + DB testing, and real-world production queries.

Topics Covered

- What is SQL Join ★
- Why SQL Joins are Important ★
- Types of SQL Joins ★
- INNER JOIN ★
- LEFT JOIN ★
- RIGHT JOIN ★
- FULL OUTER JOIN ★
- SELF JOIN ★
- CROSS JOIN ★
- Real-Time Testing Scenarios ★

► What is SQL Join

What is it: SQL Joins combine records from multiple tables using related columns.

Key Components:

- Table relationships
- Data combination
- Relational mapping

How YOU should answer: SQL Joins are used to retrieve connected data from two or more related tables using a common column.

SQL

```
SELECT e.name, d.department_name  
FROM employees e  
JOIN departments d ON e.dept_id = d.id;
```

► Types of SQL Joins

What is it: Six types of joins for different data retrieval needs.

Key Components:

- INNER JOIN — matching rows only
- LEFT JOIN — all left + matching right
- RIGHT JOIN — all right + matching left

- FULL OUTER JOIN — all rows from both
- SELF JOIN — table joined with itself
- CROSS JOIN — all possible combinations

How YOU should answer: Different SQL joins retrieve matching or non-matching data depending on the business requirement.

► Real-Time Testing Scenarios

What is it: SQL Joins in SDET validation.

Key Components:

- Validate customer and order data
- Verify user roles and permissions
- Backend API + DB validation

How YOU should answer: Joins are used to validate API responses by comparing data across related database tables.

SQL

```
-- Validate user has placed orders
SELECT u.name, COUNT(o.id) AS order_count
FROM users u LEFT JOIN orders o ON u.id = o.user_id
GROUP BY u.name;
```

Common Mistakes

- ✗ Wrong join selection
- ✗ Duplicate records from wrong join
- ✗ Not using aliases in multi-table joins

Expected Interview Questions

- What are SQL joins?
- INNER JOIN vs LEFT JOIN?
- When to use which join?
- Real-time join example?

Join Type	Returns	NULL rows
INNER JOIN	Matching rows only	None
LEFT JOIN	All left + matching right	Right side NULLs
RIGHT JOIN	All right + matching left	Left side NULLs
FULL OUTER JOIN	All rows from both	Both sides NULLs
SELF JOIN	Same table rows	Depends
CROSS JOIN	All combinations	None

INNER JOIN

20. INNER JOIN ★

Overview

INNER JOIN is one of the MOST IMPORTANT SQL concepts for retrieving matching data, backend validation, API + Database testing, and real-world business queries.

Topics Covered

- What is INNER JOIN ★
- Why INNER JOIN is Important ★
- INNER JOIN Syntax ★
- Matching Records Logic ★
- INNER JOIN with Multiple Tables ★
- INNER JOIN with WHERE Clause ★
- INNER JOIN vs LEFT JOIN ★

► What is INNER JOIN

What is it: INNER JOIN returns only records that have matching values in both tables.

Key Components:

- Matching data only
- Common records
- Excludes non-matching rows

How YOU should answer: INNER JOIN retrieves records having matching values in both joined tables. Non-matching rows are excluded.

SQL

```
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments
ON employees.dept_id = departments.id;
```

► INNER JOIN with Multiple Tables

What is it: Join more than two tables together.

Key Components:

- Multi-table joins
- Complex relationships
- Chained ON conditions

How YOU should answer: INNER JOIN can combine multiple tables by chaining multiple JOIN conditions.

SQL

```
SELECT e.name, d.dept_name, l.city
FROM employees e
INNER JOIN departments d ON e.dept_id = d.id
INNER JOIN locations l ON d.loc_id = l.id;
```

► INNER JOIN with WHERE

What is it: Filter joined data using conditions.

Key Components:

- Conditional filtering
- Data restriction

How YOU should answer: WHERE clause filters results after the JOIN operation for targeted data retrieval.

SQL

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.id
WHERE d.dept_name = 'IT';
```

► INNER JOIN vs LEFT JOIN

What is it: Key difference between the two most common joins.

Key Components:

- INNER JOIN — only matching rows
- LEFT JOIN — all left rows + matching right

How YOU should answer: INNER JOIN returns only matching rows while LEFT JOIN returns all rows from the left table.

Common Mistakes

- ✗ Missing ON condition
- ✗ Returning duplicate records
- ✗ Confusing with LEFT JOIN

Expected Interview Questions

- What is INNER JOIN?
- INNER JOIN syntax?
- INNER JOIN vs LEFT JOIN?
- Multiple table joins?

LEFT JOIN

21. LEFT JOIN ★

Overview

LEFT JOIN is one of the MOST IMPORTANT SQL concepts for retrieving all records from the left table, finding missing data, backend validation, and business reporting.

Topics Covered

- What is LEFT JOIN ★
- LEFT JOIN Syntax ★
- Matching vs Non-Matching Records ★
- LEFT JOIN with WHERE Clause ★
- Finding Missing Data ★
- LEFT JOIN vs INNER JOIN ★
- Real-Time Testing Scenarios ★

► What is LEFT JOIN

What is it: LEFT JOIN returns all records from the left table and matching records from the right table.

Key Components:

- Left table priority
- NULL for non-matching right rows
- Complete left table output

How YOU should answer: LEFT JOIN returns all rows from the left table. For non-matching rows, the right table columns show NULL.

SQL

```
SELECT employees.name, departments.dept_name
FROM employees
LEFT JOIN departments
ON employees.dept_id = departments.id;
```

► Finding Missing Data

What is it: Most powerful use case of LEFT JOIN.

Key Components:

- Find employees without department
- Detect orphan records
- Data validation

How YOU should answer: LEFT JOIN with IS NULL in WHERE clause finds records in left table with no matching right table entry.

```
SQL
-- Employees without a department
SELECT e.name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id
WHERE d.id IS NULL;
```

► Real-Time Testing Scenarios

What is it: LEFT JOIN in SDET validation.

Key Components:

- Users without orders
- Customers without transactions
- Missing relationships

How YOU should answer: LEFT JOIN is used to validate that all required relationships exist in the database.

```
SQL
-- Users without any orders
SELECT u.name
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
WHERE o.id IS NULL;
```

Common Mistakes

- ✗ Wrong filtering logic after LEFT JOIN
- ✗ Incorrect NULL handling
- ✗ Confusing LEFT and RIGHT JOIN

Expected Interview Questions

- What is LEFT JOIN?
- LEFT JOIN vs INNER JOIN?
- How to find missing records?
- Finding NULL after LEFT JOIN?

RIGHT JOIN

22. RIGHT JOIN ★

Overview

RIGHT JOIN is one of the MOST IMPORTANT SQL concepts for retrieving all records from the right table, finding unmatched data, backend validation, and business reporting.

Topics Covered

- What is RIGHT JOIN ★
- RIGHT JOIN Syntax ★
- Matching vs Non-Matching Records ★
- Finding Missing Data ★
- RIGHT JOIN vs LEFT JOIN ★
- Real-Time Testing Scenarios ★

► What is RIGHT JOIN

What is it: RIGHT JOIN returns all records from the right table and matching records from the left table.

Key Components:

- Right table priority
- NULL for non-matching left rows
- Complete right table output

How YOU should answer: RIGHT JOIN returns all rows from the right table. For non-matching rows, the left table columns show NULL.

```
SQL
SELECT employees.name, departments.dept_name
FROM employees
RIGHT JOIN departments
ON employees.dept_id = departments.id;
```

► Finding Missing Data

What is it: Find records in right table with no matching left table entry.

Key Components:

- Unmatched right records
- Data validation
- Orphan detection

How YOU should answer: RIGHT JOIN with IS NULL finds right table records that have no matching left table rows.

```
SQL
-- Departments with no employees
```

```
SELECT d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.id
WHERE e.id IS NULL;
```

► RIGHT JOIN vs LEFT JOIN

What is it: Mirror operations — same result achieved differently.

Key Components:

- RIGHT JOIN — all right rows
- LEFT JOIN — all left rows
- Can rewrite RIGHT JOIN as LEFT JOIN by swapping table order

How YOU should answer: RIGHT JOIN is the mirror of LEFT JOIN. Most developers prefer LEFT JOIN by convention.

```
SQL
-- These two are equivalent:
SELECT * FROM A RIGHT JOIN B ON A.id = B.id;
SELECT * FROM B LEFT JOIN A ON B.id = A.id;
```

Common Mistakes

- ✗ Confusing LEFT and RIGHT JOIN
- ✗ Not handling NULL values
- ✗ Wrong table ordering

Expected Interview Questions

- What is RIGHT JOIN?
- RIGHT JOIN vs LEFT JOIN?
- How to find unmatched records?
- When to use RIGHT JOIN?

FULL OUTER JOIN

23. FULL OUTER JOIN ★

Overview

FULL OUTER JOIN is one of the MOST IMPORTANT SQL concepts for retrieving all records from both tables, finding matched and unmatched data, and data reconciliation.

Topics Covered

- What is FULL OUTER JOIN ★
- FULL OUTER JOIN Syntax ★
- Matching vs Non-Matching Records ★
- Finding Missing Data ★
- FULL OUTER JOIN vs INNER JOIN ★
- Real-Time Testing Scenarios ★

► What is FULL OUTER JOIN

What is it: FULL OUTER JOIN returns all matching and non-matching records from both tables.

Key Components:

- All rows from both tables
- NULL for non-matching on either side
- Complete data visibility

How YOU should answer: FULL OUTER JOIN returns all rows from both tables. Non-matching rows show NULL for the missing side.

SQL

```
SELECT u.name, o.order_id
FROM users u
FULL OUTER JOIN orders o ON u.id = o.user_id;
```

► Finding Missing Data on Both Sides

What is it: Detect unmatched records in either table.

Key Components:

- Records with no match on either side
- Data reconciliation

How YOU should answer: FULL OUTER JOIN with IS NULL on both sides finds all unmatched records from both tables.

SQL

```
-- Find all unmatched records
SELECT u.name, o.order_id
FROM users u FULL OUTER JOIN orders o ON u.id = o.user_id
WHERE u.id IS NULL OR o.order_id IS NULL;
```

► Real-Time Testing Scenarios

What is it: FULL OUTER JOIN in SDET.

Key Components:

- Data migration validation
- Reconciliation reports
- Find all discrepancies

How YOU should answer: FULL OUTER JOIN is used to validate data migration by finding records missing on either side.

SQL

```
-- Validate data migration
SELECT old.id, new.id
FROM old_data old FULL OUTER JOIN new_data new ON old.id = new.id
WHERE old.id IS NULL OR new.id IS NULL;
```

Common Mistakes

- ✗ Confusing with INNER JOIN
- ✗ Not handling NULL on both sides
- ✗ Performance on large tables ignored

Expected Interview Questions

- What is FULL OUTER JOIN?
- FULL OUTER JOIN vs INNER JOIN?
- How to find all unmatched records?

SELF JOIN

24. SELF JOIN ★

Overview

SELF JOIN is one of the MOST IMPORTANT SQL concepts for working with hierarchical data, employee-manager relationships, and organizational structures.

Topics Covered

- What is SELF JOIN ★
- SELF JOIN Syntax ★
- Table Aliases in SELF JOIN ★
- Employee-Manager Example ★
- Hierarchical Data Handling ★
- SELF JOIN vs INNER JOIN ★
- Real-Time Testing Scenarios ★

► What is SELF JOIN

What is it: SELF JOIN joins a table with itself using table aliases.

Key Components:

- Same table used twice

- Aliases required
- Hierarchical relationships

How YOU should answer: SELF JOIN is used when records in the same table are related to each other, like employee-manager hierarchy.

SQL

```
SELECT e.name AS Employee, m.name AS Manager
FROM employees e
JOIN employees m ON e.manager_id = m.id;
```

► Employee-Manager Example

What is it: Most common SELF JOIN scenario.

Key Components:

- Employee table references itself
- manager_id points to id in same table

How YOU should answer: Employee-manager hierarchy is the classic SELF JOIN use case where the manager_id references the id column.

SQL

```
SELECT e.name AS Employee, m.name AS Manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id
ORDER BY m.name;
```

► Hierarchical Data Handling

What is it: Used for parent-child relationships.

Key Components:

- Organizational hierarchy
- Category hierarchy
- Recursive relationships

How YOU should answer: SELF JOIN handles parent-child data stored in the same table like org charts and category trees.

SQL

```
SELECT child.name AS Category, parent.name AS ParentCategory
FROM categories child
JOIN categories parent ON child.parent_id = parent.id;
```

Common Mistakes

- ✗ Not using aliases
- ✗ Confusing SELF JOIN with INNER JOIN

✗ Missing ON condition

Expected Interview Questions

- What is SELF JOIN?
- Why aliases required?
- Employee-manager SELF JOIN query?
- Hierarchical data examples?

CROSS JOIN

25. CROSS JOIN ★

Overview

CROSS JOIN is one of the MOST IMPORTANT SQL concepts for generating all possible combinations, test data creation, reporting scenarios, and understanding Cartesian Products.

Topics Covered

- What is CROSS JOIN ★
- CROSS JOIN Syntax ★
- Cartesian Product ★
- CROSS JOIN vs INNER JOIN ★
- Real-Time Use Cases ★
- Performance Considerations ★

► What is CROSS JOIN

What is it: CROSS JOIN returns every possible combination of rows between two tables.

Key Components:

- All combinations
- No ON condition required
- Cartesian product result

How YOU should answer: CROSS JOIN combines every row from the first table with every row from the second table.

SQL

```
SELECT products.name, sizes.size
FROM products
CROSS JOIN sizes;
```

► Cartesian Product

What is it: The result generated by CROSS JOIN.

Key Components:

- Total rows = Table A rows × Table B rows
- Can generate huge datasets

How YOU should answer: CROSS JOIN generates a Cartesian product — if Table A has 5 rows and Table B has 3, result has 15 rows.

SQL

```
-- 5 products × 3 sizes = 15 combinations  
SELECT p.name, s.size FROM products p CROSS JOIN sizes s;
```

► Real-Time Testing Scenarios

What is it: CROSS JOIN in SDET testing.

Key Components:

- Generate test data combinations
- Product-size combinations
- Test all scenarios

How YOU should answer: CROSS JOIN is used to generate comprehensive test data covering all possible combinations.

SQL

```
-- Generate all test combinations  
SELECT env.name, browser.name  
FROM environments env CROSS JOIN browsers browser;
```

Common Mistakes

- ✗ Huge unintended datasets
- ✗ Using CROSS JOIN accidentally without ON clause
- ✗ Performance issues on large tables

Expected Interview Questions

- What is CROSS JOIN?
- What is Cartesian Product?
- CROSS JOIN vs INNER JOIN?
- Real-time use cases?

★ SQL Joins — Final Interview Gold

1. INNER JOIN — matching rows only
2. LEFT JOIN — find missing/orphan records with IS NULL
3. RIGHT JOIN — mirror of LEFT JOIN
4. FULL OUTER JOIN — data reconciliation

- 5. SELF JOIN — employee-manager hierarchy
- 6. CROSS JOIN — Cartesian product / test data generation

GROUP BY CLAUSE

26. GROUP BY Clause ★

Overview

GROUP BY is one of the MOST IMPORTANT SQL concepts for grouping similar records, data summarization, reporting and analytics, and backend validation.

Topics Covered

- What is GROUP BY ★
- Why GROUP BY is Important ★
- GROUP BY Syntax ★
- GROUP BY with Aggregate Functions ★
- GROUP BY with Multiple Columns ★
- GROUP BY vs ORDER BY ★
- Real-Time Testing Scenarios ★

► What is GROUP BY

What is it: GROUP BY groups rows based on one or more columns so aggregate functions can calculate values for each group.

Key Components:

- Groups similar records
- Used with aggregate functions
- Creates summarized output

How YOU should answer: GROUP BY groups rows based on specified columns before applying aggregate functions.

SQL

```
SELECT department, COUNT(*) AS total_employees
FROM employees
GROUP BY department;
```

► GROUP BY with Aggregate Functions

What is it: Most common GROUP BY use case.

Key Components:

- COUNT() — count per group

- SUM() — total per group
- AVG() — average per group
- MIN()/MAX() — extremes per group

How YOU should answer: Aggregate functions calculate values for each group created by GROUP BY.

SQL

```
SELECT department,
COUNT(*) AS total,
SUM(salary) AS total_salary,
AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

► GROUP BY with Multiple Columns

What is it: Group by more than one column.

Key Components:

- Multiple grouping columns
- Finer granularity

How YOU should answer: Multiple columns in GROUP BY creates groups for each unique combination of those columns.

SQL

```
SELECT department, city, COUNT(*) AS total
FROM employees
GROUP BY department, city;
```

► GROUP BY vs ORDER BY

What is it: Key difference between grouping and sorting.

Key Components:

- GROUP BY — groups rows, reduces result set
- ORDER BY — sorts rows, keeps all rows

How YOU should answer: GROUP BY groups rows for aggregation while ORDER BY sorts the final result set.

SQL

```
SELECT department, COUNT(*) AS total
FROM employees
GROUP BY department
ORDER BY total DESC;
```

Common Mistakes

- ✗ Selecting non-grouped columns without aggregate
- ✗ Forgetting GROUP BY with aggregates
- ✗ Confusing GROUP BY with ORDER BY

Expected Interview Questions

- What is GROUP BY?
- GROUP BY vs ORDER BY?
- GROUP BY with COUNT example?
- GROUP BY multiple columns?

HAVING CLAUSE

27. HAVING Clause ★

Overview

HAVING is one of the MOST IMPORTANT SQL concepts for filtering grouped data, aggregate function filtering, business reports and dashboards, and backend validation queries.

Topics Covered

- What is HAVING ★
- Why HAVING is Important ★
- HAVING Syntax ★
- HAVING with Aggregate Functions ★
- HAVING vs WHERE ★
- HAVING with GROUP BY ★
- Real-Time Testing Scenarios ★

► What is HAVING

What is it: HAVING filters grouped records after aggregation.

Key Components:

- Filters grouped data
- Works with aggregate functions
- Used after GROUP BY

How YOU should answer: HAVING filters grouped records after GROUP BY, whereas WHERE filters individual rows before grouping.

SQL

```
SELECT department, COUNT(*) AS total
FROM employees
GROUP BY department
```

```
HAVING COUNT(*) > 5;
```

► HAVING vs WHERE

What is it: Most important HAVING interview question.

Key Components:

- WHERE — filters rows BEFORE grouping
- HAVING — filters groups AFTER aggregation
- WHERE cannot use aggregate functions
- HAVING can use aggregate functions

How YOU should answer: Use WHERE to filter rows before grouping and HAVING to filter groups after aggregation.

```
SQL
SELECT department, AVG(salary) AS avg_sal
FROM employees
WHERE status = 'ACTIVE'
GROUP BY department
HAVING AVG(salary) > 60000;
```

► Real-Time Testing Scenarios

What is it: HAVING in SDET validation.

Key Components:

- Departments with more than 10 employees
- Products with more than 500 orders

How YOU should answer: HAVING is used to validate business rules on grouped data in backend testing.

```
SQL
-- Validate high-value customers
SELECT customer_id, SUM(amount) AS total
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 10000;
```

Common Mistakes

- ✗ Using HAVING instead of WHERE for row filtering
- ✗ Forgetting GROUP BY before HAVING
- ✗ Using HAVING without aggregate function

Expected Interview Questions

- What is HAVING?

- HAVING vs WHERE?
- Why HAVING used?
- Can HAVING work without GROUP BY?

Feature	WHERE	HAVING
Filters	Individual rows	Grouped rows
When	Before GROUP BY	After GROUP BY
Aggregates	Cannot use	Can use
Performance	More efficient	Less efficient

AGGREGATE FUNCTIONS

28. Aggregate Functions ★

Overview

Aggregate Functions are one of the MOST IMPORTANT SQL concepts for data summarization, business reporting, dashboard analytics, and backend validation queries.

Topics Covered

- What are Aggregate Functions ★
- COUNT() Function ★
- SUM() Function ★
- AVG() Function ★
- MIN() and MAX() Functions ★
- Aggregate Functions with GROUP BY ★
- NULL Handling in Aggregates ★

► What are Aggregate Functions

What is it: Aggregate functions perform calculations on multiple rows and return a single summarized result.

Key Components:

- COUNT() — count rows
- SUM() — total values
- AVG() — average
- MIN() — minimum
- MAX() — maximum

How YOU should answer: Aggregate functions perform calculations on a set of rows and return a single value.

```
SELECT COUNT(*), SUM(salary), AVG(salary), MIN(salary), MAX(salary)
FROM employees;
```

► COUNT Function

What is it: Returns the number of rows.

Key Components:

- COUNT(*) — counts all rows including NULL
- COUNT(column) — counts non-NULL values only

How YOU should answer: COUNT(*) counts all rows while COUNT(column) ignores NULL values.

```
SQL
SELECT COUNT(*) AS total_rows FROM employees;
SELECT COUNT(email) AS with_email FROM employees;
```

► SUM and AVG Functions

What is it: Calculate totals and averages.

Key Components:

- SUM() — total of numeric column
- AVG() — average ignoring NULL

How YOU should answer: SUM() returns total of all values and AVG() returns average ignoring NULL values.

```
SQL
SELECT SUM(salary) AS total_payroll, AVG(salary) AS avg_salary
FROM employees;
```

► MIN and MAX Functions

What is it: Find extreme values.

Key Components:

- MIN() — smallest value
- MAX() — largest value

How YOU should answer: MIN() returns the smallest value and MAX() returns the largest value in a column.

```
SQL
SELECT MIN(salary) AS lowest, MAX(salary) AS highest FROM employees;
```

► Aggregate with GROUP BY

What is it: Most common usage pattern.

Key Components:

- Per-group calculations

- Department-wise reports

How YOU should answer: Aggregate functions with GROUP BY calculate values for each group independently.

SQL

```
SELECT department, COUNT(*) AS total, AVG(salary) AS avg_sal  
FROM employees GROUP BY department;
```

Common Mistakes

- ✗ Confusing COUNT(*) and COUNT(column)
- ✗ Using SUM() on non-numeric columns
- ✗ Forgetting GROUP BY

Expected Interview Questions

- COUNT(*) vs COUNT(column)?
- How does AVG handle NULL?
- Aggregate with GROUP BY example?
- MIN vs MAX vs ORDER BY?

★ Aggregate Functions Interview Gold

1. COUNT(*) vs COUNT(column) — NULL handling
2. AVG ignores NULL — verify with COALESCE if needed
3. SUM only works on numeric columns
4. Always use GROUP BY with aggregates on grouped data
5. HAVING filters aggregate results

UNION vs UNION ALL

29. UNION vs UNION ALL ★

Overview

UNION and UNION ALL are one of the MOST IMPORTANT SQL concepts for combining data from multiple tables, report generation, data migration, and backend validation.

Topics Covered

- What is UNION ★
- What is UNION ALL ★
- UNION Syntax ★

- UNION ALL Syntax ★
- UNION vs UNION ALL ★
- Performance Considerations ★
- Real-Time Testing Scenarios ★

► What is UNION

What is it: UNION combines the results of multiple SELECT queries and removes duplicate rows.

Key Components:

- Combines multiple result sets
- Removes duplicates
- Same column count required

How YOU should answer: UNION combines results of multiple SELECT queries into a single result set while removing duplicate rows.

```
SQL
SELECT email FROM customers
UNION
SELECT email FROM newsletter_users;
```

► What is UNION ALL

What is it: UNION ALL combines multiple SELECT statements without removing duplicate records.

Key Components:

- Combines result sets
- Keeps all duplicates
- Faster than UNION

How YOU should answer: UNION ALL returns all rows including duplicates and is generally faster than UNION.

```
SQL
SELECT email FROM customers
UNION ALL
SELECT email FROM newsletter_users;
```

► UNION vs UNION ALL

What is it: Key difference.

Key Components:

- UNION — removes duplicates, slightly slower
- UNION ALL — keeps duplicates, faster
- Use UNION ALL when duplicates are acceptable

How YOU should answer: UNION removes duplicate rows whereas UNION ALL returns all rows including duplicates and is generally faster.

SQL

```
-- UNION: unique customer list
-- UNION ALL: complete transaction history
```

► Real-Time Testing Scenarios

What is it: UNION in SDET automation.

Key Components:

- Merge data from multiple environments
- Combine test execution logs

How YOU should answer: UNION is used to combine execution results from multiple test environments for validation.

SQL

```
SELECT test_name, result FROM qa_results
UNION ALL
SELECT test_name, result FROM uat_results;
```

Common Mistakes

- ✗ Different column count in queries
- ✗ Incompatible data types
- ✗ Using UNION when UNION ALL is sufficient

Expected Interview Questions

- What is UNION?
- UNION vs UNION ALL?
- Which is faster?
- When to use UNION ALL?

Feature	UNION	UNION ALL
Duplicates	Removed	Kept
Performance	Slower	Faster
Use case	Unique results	All results
Sort	Implicit sort	No sort

SUBQUERIES

30. Subqueries ★

Overview

Subqueries are one of the MOST IMPORTANT SQL concepts for writing complex SQL queries, backend data validation, nested data retrieval, and product company interviews.

Topics Covered

- What is a Subquery ★
- Types of Subqueries ★
- Subquery Syntax ★
- Subquery with WHERE Clause ★
- Correlated vs Non-Correlated Subqueries ★
- Performance Considerations ★

► What is a Subquery

What is it: A subquery is a query nested inside another SQL query that provides data to the outer query.

Key Components:

- Nested query
- Inner query executes first
- Outer query uses result

How YOU should answer: A subquery is an inner query nested inside another query. The inner query executes first and returns data to the outer query.

```
SQL
SELECT name FROM employees
WHERE department_id = (
  SELECT id FROM departments WHERE name = 'IT'
);
```

► Types of Subqueries

What is it: Different subquery types based on behavior.

Key Components:

- Single-row subquery — returns one row
- Multiple-row subquery — returns multiple rows
- Correlated subquery — depends on outer query
- Non-correlated subquery — independent of outer query

How YOU should answer: SQL supports different types of subqueries depending on how many rows they return and whether they depend on the outer query.

► Subquery with WHERE

What is it: Most common subquery use case.

Key Components:

- Dynamic filtering
- Nested conditions

How YOU should answer: Subqueries inside WHERE clauses dynamically filter data based on another query result.

```
SQL
-- Employees earning above average
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

► Correlated vs Non-Correlated

What is it: Important difference for interviews.

Key Components:

- Non-correlated — executes once, independent
- Correlated — executes once per outer row, references outer query

How YOU should answer: Non-correlated subqueries execute once while correlated subqueries execute for each row of the outer query.

```
SQL
-- Non-correlated (executes once)
SELECT name FROM employees
WHERE dept_id = (SELECT id FROM departments WHERE name = 'IT');

-- Correlated (executes per row)
SELECT name FROM employees e1
WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = e1.dept_id);
```

Common Mistakes

- ✗ Missing parentheses
- ✗ Using = instead of IN for multi-row subqueries
- ✗ Overusing correlated subqueries

Expected Interview Questions

- What is a subquery?
- Types of subqueries?
- Correlated vs Non-Correlated?
- Subquery vs JOIN?

EXISTS vs IN

31. EXISTS vs IN ★

Overview

EXISTS and IN are one of the MOST IMPORTANT SQL concepts for filtering data using subqueries, backend database validation, and optimizing SQL queries.

Topics Covered

- What is EXISTS ★
- What is IN ★
- EXISTS Syntax ★
- IN Syntax ★
- EXISTS vs IN ★
- Performance Comparison ★
- Real-Time Testing Scenarios ★

► What is EXISTS

What is it: EXISTS checks whether the subquery returns any rows — returns TRUE or FALSE.

Key Components:

- Returns TRUE/FALSE
- Stops after first match
- Used with correlated subqueries

How YOU should answer: EXISTS checks whether the subquery returns any rows. If at least one row is found, it returns TRUE.

```
SQL
SELECT name FROM employees e
WHERE EXISTS (
  SELECT 1 FROM orders o WHERE o.emp_id = e.id
);
```

► What is IN

What is it: IN checks whether a value exists in a list or in the result of a subquery.

Key Components:

- Membership operator
- Multiple values check
- Works with subqueries

How YOU should answer: IN checks whether a column value matches any value in a specified list or subquery result.

```
SQL
SELECT name FROM employees
WHERE department_id IN (SELECT id FROM departments WHERE location = 'Bangalore');
```

► EXISTS vs IN

What is it: Key performance and behavior differences.

Key Components:

- EXISTS — stops after first match, better for large datasets
- IN — evaluates all returned values
- EXISTS — returns TRUE/FALSE
- IN — returns actual matching rows

How YOU should answer: EXISTS is generally more efficient for large datasets because it stops after finding the first matching row.

SQL

```
-- EXISTS: check if employee has orders
-- IN: check if dept_id in a list
```

► **Real-Time Testing Scenarios**

What is it: EXISTS and IN in SDET testing.

Key Components:

- Validate record existence after API call
- Check data in expected list

How YOU should answer: EXISTS validates that records were created by API operations and IN validates values match expected lists.

SQL

```
-- Validate record created by API
SELECT * FROM users
WHERE EXISTS (SELECT 1 FROM audit_log WHERE action = 'CREATE' AND user_id =
users.id);
```

Common Mistakes

- ✗ Confusing EXISTS with IN
- ✗ Using IN for very large datasets
- ✗ Misunderstanding EXISTS return value

Expected Interview Questions

- What is EXISTS?
- EXISTS vs IN?
- Which performs better?
- When to use EXISTS vs IN?

Feature	EXISTS	IN
Returns	TRUE/FALSE	Matching rows
Stops at	First match	After all values

Performance	Better for large data	Fine for small lists
NULL handling	Not affected	Affected by NULL
Subquery type	Correlated usually	Non-correlated

CASE STATEMENT

32. CASE Statement ★

Overview

CASE Statement is one of the MOST IMPORTANT SQL concepts for conditional data transformation, business reporting, dashboard generation, and backend validation queries.

Topics Covered

- What is CASE Statement ★
- Simple CASE Statement ★
- Searched CASE Statement ★
- CASE with ORDER BY ★
- CASE with Aggregate Functions ★
- Real-Time Testing Scenarios ★

► What is CASE Statement

What is it: CASE Statement allows conditional logic in SQL, similar to IF-ELSE in programming.

Key Components:

- IF-ELSE like logic
- Conditional expressions
- Multiple conditions
- WHEN/THEN/ELSE/END

How YOU should answer: CASE evaluates conditions sequentially and returns the value for the first matching condition.

```
SQL
SELECT name,
CASE
WHEN salary >= 100000 THEN 'High'
WHEN salary >= 50000 THEN 'Medium'
ELSE 'Low'
END AS salary_band
FROM employees;
```

► Simple vs Searched CASE

What is it: Two forms of CASE statement.

Key Components:

- Simple CASE — compares one expression to multiple values
- Searched CASE — evaluates multiple independent conditions

How YOU should answer: Simple CASE compares a single expression while Searched CASE supports complex multi-condition logic.

```
SQL
-- Simple CASE
CASE status
WHEN 1 THEN 'Active'
WHEN 0 THEN 'Inactive'
END

-- Searched CASE
CASE
WHEN salary > 100000 THEN 'High'
ELSE 'Low'
END
```

► CASE with Aggregate Functions

What is it: Conditional aggregation pattern.

Key Components:

- Conditional SUM
- Conditional COUNT
- Pivot-like reports

How YOU should answer: CASE can be combined with aggregate functions to perform conditional calculations.

```
SQL
SELECT
SUM(CASE WHEN dept = 'IT' THEN salary ELSE 0 END) AS it_payroll,
SUM(CASE WHEN dept = 'HR' THEN salary ELSE 0 END) AS hr_payroll
FROM employees;
```

► Real-Time Testing Scenarios

What is it: CASE in SDET validation.

Key Components:

- Categorize test results

- Pass/Fail reports
- Status mapping

How YOU should answer: CASE is used to generate Pass/Fail execution reports and categorize test results.

SQL

```
SELECT test_name,  
CASE  
WHEN result = 1 THEN 'PASS'  
ELSE 'FAIL'  
END AS status  
FROM test_results;
```

Common Mistakes

- ✗ Incorrect condition ordering
- ✗ Missing END keyword
- ✗ Using comparison operators in Simple CASE

Expected Interview Questions

- What is CASE statement?
- Simple vs Searched CASE?
- CASE with aggregate functions?
- Real-time CASE example?

VIEWS IN SQL

33. Views in SQL ★

Overview

Views are one of the MOST IMPORTANT SQL concepts for simplifying complex SQL queries, data security, reusable database queries, and backend validation.

Topics Covered

- What is a View ★
- Why Views are Important ★
- CREATE VIEW Syntax ★
- Updating Views ★
- Advantages of Views ★
- Views vs Tables ★
- Real-Time Testing Scenarios ★

► What is a View

What is it: A View is a virtual table whose data comes from one or more underlying tables.

Key Components:

- Virtual table
- Stores only SQL query
- No physical data storage
- Reusable

How YOU should answer: A View stores a SELECT query definition, not actual data. Querying a view executes the underlying query.

SQL

```
CREATE VIEW active_employees AS
SELECT id, name, salary FROM employees
WHERE status = 'ACTIVE';

-- Query the view
SELECT * FROM active_employees;
```

► Why Views are Important

What is it: Views improve security, readability, and reusability.

Key Components:

- Security — hide sensitive columns
- Simplicity — hide complex joins
- Reusability — reuse complex queries
- Abstraction — hide implementation

How YOU should answer: Views simplify frequently used complex queries, improve security by restricting column access, and provide reusability.

► Advantages of Views

What is it: Key benefits of using Views.

Key Components:

- Hide sensitive data (e.g. salary, SSN)
- Simplify complex multi-table queries
- Provide consistent interface for queries
- Reduce code duplication

How YOU should answer: Views provide security by exposing only necessary columns and simplify complex queries into reusable objects.

SQL

```
-- Secure view – no salary column
CREATE VIEW employee_public AS
```

```
SELECT id, name, department FROM employees;
```

► Views vs Tables

What is it: Key differences.

Key Components:

- View — virtual, stores query, no direct data
- Table — physical, stores actual data

How YOU should answer: Views store query definitions while tables store actual data physically.

Common Mistakes

- ✗ Assuming Views always improve performance
- ✗ Trying to update complex views
- ✗ Confusing view with materialized view

Expected Interview Questions

- What is a View?
- How to create a View?
- View vs Table?
- Can Views be updated?
- Security benefits of Views?

Feature	View	Table
Stores	SQL query	Actual data
Physical storage	No	Yes
Performance	Executes on query	Direct access
Updateable	Sometimes	Yes
Security	Can restrict columns	Full access

INDEXES IN SQL

34. Indexes in SQL ★

Overview

Indexes are one of the MOST IMPORTANT SQL concepts for improving query performance, faster data retrieval, optimizing backend applications, and product company interviews.

Topics Covered

- What is an Index ★

- Why Indexes are Important ★
- CREATE INDEX Syntax ★
- Types of Indexes ★
- Clustered vs Non-Clustered Index ★
- Advantages and Disadvantages ★
- Real-Time Testing Scenarios ★

► What is an Index

What is it: An Index is a database object that improves the speed of data retrieval operations.

Key Components:

- Faster searching
- Improves SELECT performance
- Built on columns
- Similar to book index

How YOU should answer: An Index creates a lookup structure on a column allowing the database to find rows faster without scanning the entire table.

SQL

```
CREATE INDEX idx_employee_name ON employees(name);  
CREATE INDEX idx_order_date ON orders(order_date);
```

► Types of Indexes

What is it: Different index types for different scenarios.

Key Components:

- Primary Index — auto-created with Primary Key
- Unique Index — enforces unique values
- Composite Index — multiple columns
- Clustered Index — sorts actual table data (one per table)
- Non-Clustered Index — separate lookup structure (multiple allowed)

How YOU should answer: SQL supports different index types depending on uniqueness requirements and column combinations.

SQL

```
CREATE UNIQUE INDEX idx_email ON users(email);  
CREATE INDEX idx_composite ON orders(customer_id, order_date);
```

► Clustered vs Non-Clustered

What is it: Most asked index interview question.

Key Components:

- Clustered — sorts actual table data, one per table, fastest range queries
- Non-Clustered — separate structure, multiple allowed, slightly slower

How YOU should answer: Clustered index determines the physical order of data in the table while Non-clustered creates a separate structure.

► Advantages and Disadvantages

What is it: Tradeoffs of using indexes.

Key Components:

- Advantages: faster SELECT, better search, improved JOIN performance
- Disadvantages: extra storage, slower INSERT/UPDATE/DELETE, maintenance needed

How YOU should answer: Indexes improve read performance but slow down write operations and require additional storage.

Common Mistakes

- ✗ Over-indexing all columns
- ✗ Ignoring write performance impact
- ✗ Never reviewing unused indexes

Expected Interview Questions

- What is an Index?
- Clustered vs Non-Clustered?
- When to create indexes?
- Index advantages and disadvantages?

Index Type	Uniqueness	Per Table	Best For
Clustered	Not required	One only	Range queries
Non-Clustered	Not required	Multiple	Point lookups
Unique	Enforced	Multiple	Duplicate prevention
Composite	Optional	Multiple	Multi-column queries

STORED PROCEDURES

35. Stored Procedures ★

Overview

Stored Procedures are one of the MOST IMPORTANT SQL concepts for reusable database logic, backend automation, business rule implementation, and product company interviews.

Topics Covered

- What is a Stored Procedure ★
- Why Stored Procedures are Important ★

- CREATE PROCEDURE Syntax ★
- Executing Stored Procedures ★
- Parameters in Stored Procedures ★
- Advantages and Disadvantages ★
- Real-Time Testing Scenarios ★

► What is a Stored Procedure

What is it: A Stored Procedure is a precompiled set of SQL statements stored in the database.

Key Components:

- Stored inside database
- Reusable
- Precompiled
- Can accept parameters
- Improves maintainability

How YOU should answer: A Stored Procedure is a saved SQL block that can be executed whenever needed without rewriting the SQL.

SQL

```
CREATE PROCEDURE GetActiveEmployees
AS
BEGIN
SELECT * FROM employees WHERE status = 'ACTIVE';
END;
```

► CREATE PROCEDURE Syntax

What is it: Standard syntax for creating a Stored Procedure.

Key Components:

- CREATE PROCEDURE keyword
- BEGIN and END block
- SQL statements inside

How YOU should answer: A stored procedure is created using CREATE PROCEDURE with SQL logic inside BEGIN...END block.

SQL

```
CREATE PROCEDURE GetEmployeeById
@EmpId INT
AS
BEGIN
SELECT * FROM employees WHERE employee_id = @EmpId;
END;
```

► Executing Stored Procedures

What is it: How to call a stored procedure.

Key Components:

- EXEC or EXECUTE keyword
- Pass parameters
- Return results

How YOU should answer: Stored procedures are called using EXEC or EXECUTE with parameter values.

SQL

```
EXEC GetActiveEmployees;
```

```
EXEC GetEmployeeById 101;
```

► Advantages and Disadvantages

What is it: Tradeoffs of stored procedures.

Key Components:

- Advantages: reusability, performance, security, reduced network traffic
- Disadvantages: harder debugging, vendor-specific syntax, maintenance complexity

How YOU should answer: Stored procedures improve performance and maintainability but may increase database dependency.

Common Mistakes

- ✗ Forgetting BEGIN and END
- ✗ Using SELECT instead of EXEC
- ✗ Hardcoding values inside procedures

Expected Interview Questions

- What is a Stored Procedure?
- CREATE PROCEDURE syntax?
- EXEC vs EXECUTE?
- Procedure vs Function?
- Real-time usage?

TRANSACTIONS IN SQL

36. Transactions in SQL ★

Overview

Transactions are one of the MOST IMPORTANT SQL concepts for maintaining data consistency, ensuring data integrity, banking and financial applications, and product company interviews.

Topics Covered

- What is a Transaction ★
- Why Transactions are Important ★
- Transaction Syntax ★
- ACID Properties ★
- Transaction States ★
- Transaction Control Commands ★
- Real-Time Testing Scenarios ★

► What is a Transaction

What is it: A transaction is a logical unit of work containing one or more SQL statements.

Key Components:

- Single unit of work
- Multiple SQL statements
- All-or-nothing execution
- Supports COMMIT and ROLLBACK

How YOU should answer: A transaction groups multiple SQL operations into a single unit — either all operations succeed or none are applied.

SQL

```
BEGIN TRANSACTION;  
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;  
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;  
COMMIT;
```

► ACID Properties

What is it: Four properties that guarantee reliable transactions.

Key Components:

- A — Atomicity: all or nothing
- C — Consistency: valid state before and after
- I — Isolation: transactions independent of each other
- D — Durability: committed changes are permanent

How YOU should answer: ACID properties ensure transactions are processed reliably maintaining data integrity.

► Transaction Control Commands

What is it: Commands to control transaction execution.

Key Components:

- BEGIN TRANSACTION — starts transaction
- COMMIT — saves all changes permanently
- ROLLBACK — undoes all changes
- SAVEPOINT — creates checkpoint for partial rollback

How YOU should answer: Transaction control commands manage the execution, saving, and recovery of database transactions.

```
SQL
BEGIN TRANSACTION;
-- SQL operations here
SAVEPOINT sp1;
-- More operations
ROLLBACK TO sp1; -- partial rollback
COMMIT;
```

► Real-Time Testing Scenarios

What is it: Transactions in SDET testing.

Key Components:

- Verify DB rollback when API fails
- Banking transfer validation
- Order placement atomicity

How YOU should answer: Transactions are validated to ensure partial failures are rolled back and successful operations are committed.

```
SQL
-- Validate transaction rollback on API failure
BEGIN TRANSACTION;
INSERT INTO orders VALUES (...);
-- Simulate failure
ROLLBACK;
-- Verify: record should not exist
SELECT * FROM orders WHERE id = ?;
```

Common Mistakes

- ✗ Forgetting to COMMIT
- ✗ Not handling rollback on failure
- ✗ Long-running open transactions

Expected Interview Questions

- What is a transaction?
- What are ACID properties?

- What is Atomicity?
- What is Isolation?
- SAVEPOINT usage?

COMMIT / ROLLBACK

37. COMMIT / ROLLBACK ★

Overview

COMMIT and ROLLBACK are one of the MOST IMPORTANT SQL TCL commands for saving database changes, undoing failed transactions, maintaining data consistency, and backend validation.

Topics Covered

- What is COMMIT ★
- What is ROLLBACK ★
- Why COMMIT and ROLLBACK are Important ★
- COMMIT Syntax ★
- ROLLBACK Syntax ★
- COMMIT vs ROLLBACK ★
- SAVEPOINT ★
- Real-Time Testing Scenarios ★

► What is COMMIT

What is it: COMMIT permanently saves all changes made during the current transaction.

Key Components:

- Saves changes permanently
- Ends the transaction
- Cannot be undone by ROLLBACK
- Makes data visible to other users

How YOU should answer: COMMIT permanently saves transaction changes to the database. After COMMIT, changes cannot be undone.

SQL

```
BEGIN TRANSACTION;  
INSERT INTO users(name) VALUES('John');  
COMMIT;
```

► What is ROLLBACK

What is it: ROLLBACK cancels all changes made during the current transaction before COMMIT.

Key Components:

- Cancels all changes
- Restores previous state
- Used after errors
- Cannot undo after COMMIT

How YOU should answer: ROLLBACK undoes all uncommitted changes and restores the database to its previous consistent state.

```
SQL
BEGIN TRANSACTION;
DELETE FROM employees WHERE employee_id = 5;
ROLLBACK; -- Cancels the delete
```

► COMMIT vs ROLLBACK

What is it: Key differences.

Key Components:

- COMMIT — makes changes permanent
- ROLLBACK — undoes all uncommitted changes
- ROLLBACK cannot undo after COMMIT

How YOU should answer: COMMIT permanently saves transaction changes whereas ROLLBACK discards all uncommitted changes.

```
SQL
-- Successful payment
BEGIN TRANSACTION;
-- payment SQL
COMMIT;

-- Failed payment
BEGIN TRANSACTION;
-- payment SQL
ROLLBACK;
```

► SAVEPOINT

What is it: Creates a checkpoint for partial rollback.

Key Components:

- Checkpoint in transaction
- Partial rollback possible
- ROLLBACK TO savepoint_name

How YOU should answer: SAVEPOINT allows rolling back to a specific point instead of cancelling the entire transaction.

```
SQL
```

```
BEGIN TRANSACTION;
UPDATE employees SET salary = salary * 1.1;
SAVEPOINT after_salary_update;
DELETE FROM temp_table;
ROLLBACK TO after_salary_update; -- only undoes DELETE
COMMIT; -- commits salary update
```

► Real-Time Testing Scenarios

What is it: COMMIT/ROLLBACK in SDET testing.

Key Components:

- Verify DB state after successful API call → COMMIT
- Verify DB rollback when API fails → ROLLBACK
- Test partial failures with SAVEPOINT

How YOU should answer: COMMIT and ROLLBACK are validated to ensure data integrity after API operations succeed or fail.

```
SQL
-- After successful order API → verify COMMIT
SELECT * FROM orders WHERE id = ?;

-- After failed payment API → verify ROLLBACK
SELECT COUNT(*) FROM payments WHERE id = ?;

-- Should return 0 if properly rolled back
```

Common Mistakes

- ✗ Trying to ROLLBACK after COMMIT
- ✗ Not using transactions in critical operations
- ✗ Ignoring transaction testing

Expected Interview Questions

- What is COMMIT?
- What is ROLLBACK?
- COMMIT vs ROLLBACK?
- What is SAVEPOINT?
- Can ROLLBACK undo COMMIT?

Command	Action	Can Undo?
COMMIT	Save changes permanently	No
ROLLBACK	Undo all uncommitted changes	N/A (already undone)
SAVEPOINT	Create rollback checkpoint	Yes (partial)

ROLLBACK TO	Undo to savepoint	Yes
-------------	-------------------	-----

★ Final Interview Gold — Part 2

1. SQL Joins — INNER vs LEFT vs RIGHT vs FULL OUTER
2. GROUP BY + HAVING — grouping and filtering aggregates
3. HAVING vs WHERE — filter after vs before GROUP BY
4. Subqueries — correlated vs non-correlated
5. EXISTS vs IN — performance difference
6. CASE Statement — conditional logic in SQL
7. Indexes — clustered vs non-clustered tradeoffs
8. Transactions — ACID properties + COMMIT/ROLLBACK